

# FANTASTIC HEAPS

---

... and where to find them.  
Datastructure Love!

Florian Sihler

September 21, 2026  
SP, Ulm University

# FANTASTIC HEAPS

---

... and where to find them.  
Datastructure Love!

Florian Sihler



Officially supported by the Pingu-Foundation for Emotional Support. There will be light and there will be cute waddlers!

September 21, 2026  
SP, Ulm University

## 2.1

# Disclaimer

[6]: *Episode-Recursion* Sihler, 2021

[5]: *BTX-Package*, *tikzpingus*  
Sihler, 2021



**Preliminaries**



Kinds of Heaps



Operations



Arrays



Wrap-Up



## 2.2

# Disclaimer

- This “heap” has nothing to do with the one from the recursion episode!

[6]: *Episode-Recursion* Sihler, 2021

[5]: *BTXK-Package*, tikzpingus  
Sihler, 2021



## 2.3

# Disclaimer

- This “heap” has nothing to do with the one from the recursion episode!
- We restrict ourselves to heap-*trees*

[6]: *Episode-Recursion* Sihler, 2021

[5]: *BTX-Package*, tikzpingus  
Sihler, 2021



## 2.4

# Disclaimer

- This “heap” has nothing to do with the one from the recursion episode!
- We restrict ourselves to heap-*trees*
- We ignore rotations

[6]: *Episode-Recursion* Sihler, 2021

[5]: *BTk-Package*, tikzpingus  
Sihler, 2021

## Disclaimer

- This “heap” has nothing to do with the one from the recursion episode!
- We restrict ourselves to heap-*trees*
- We ignore rotations
- These slides were made in  $\sim 3.5$  hours<sup>hence condensed</sup>

[6]: *Episode-Recursion* Sihler, 2021

[5]: *BTPK-Package*, *tikzpingus*  
Sihler, 2021

## 3.1

# The Poster Children



Preliminaries



**Kinds of Heaps**



Operations



Arrays

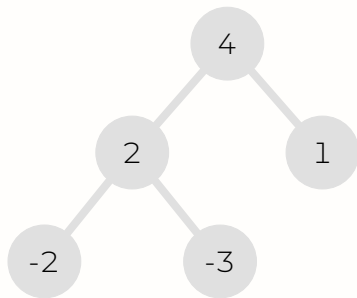


Wrap-Up



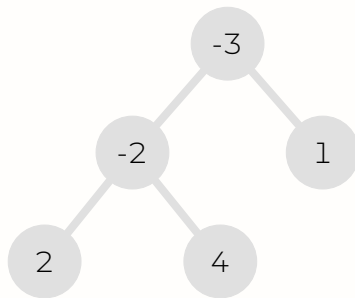
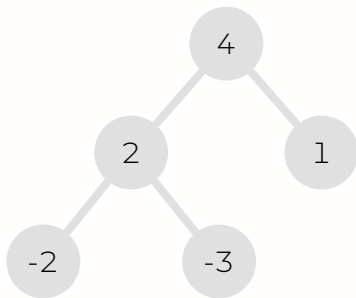
## 3.2

# The Poster Children



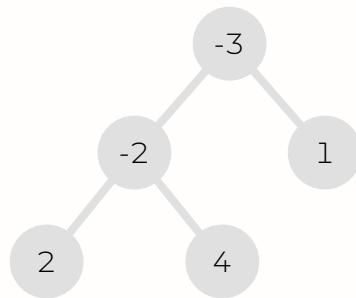
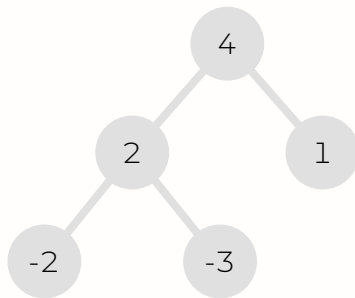
### 3.3

## The Poster Children



### 3.4

## The Poster Children



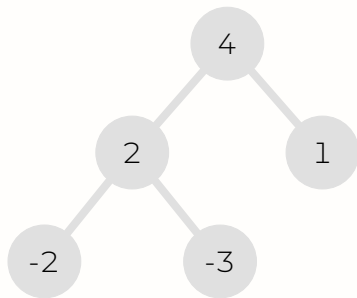
**Max-Heap**

parents  $\geq$  children



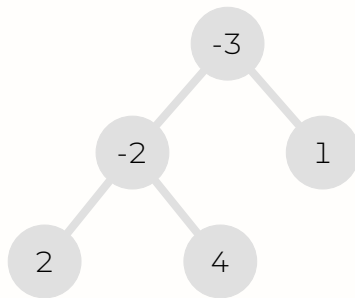
### 3.5

## The Poster Children



**Max-Heap**

parents  $\geq$  children



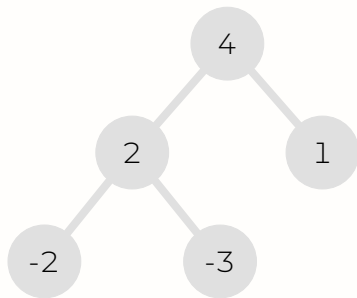
**Min-Heap**

parents  $\leq$  children

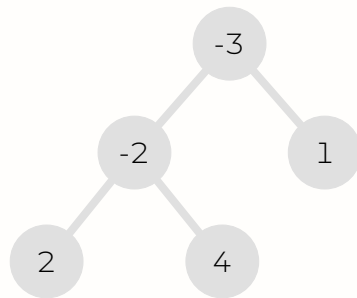


## 3.6

# The Poster Children



- **Max-Heap**  
parents  $\geq$  children



- Min-Heap**  
parents  $\leq$  children

## 4.1

# The Anatomy



Preliminaries



**Kinds of Heaps**



Operations



Arrays

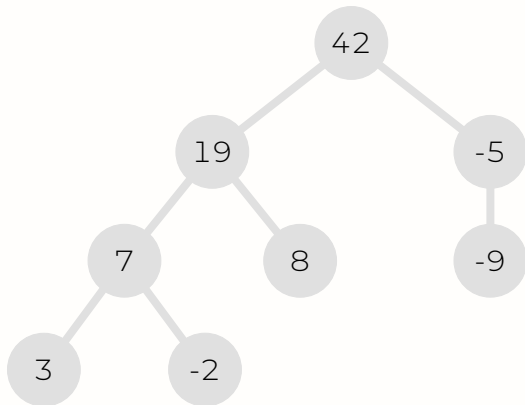


Wrap-Up



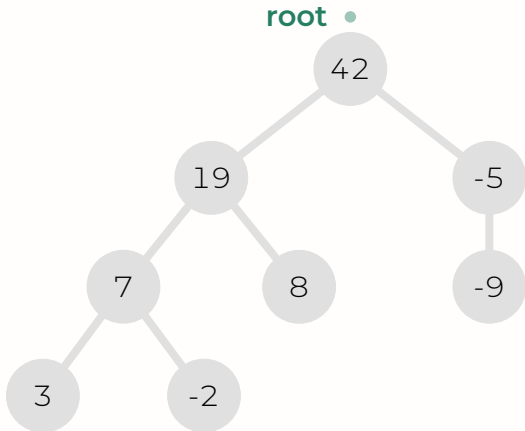
## 4.2

# The Anatomy



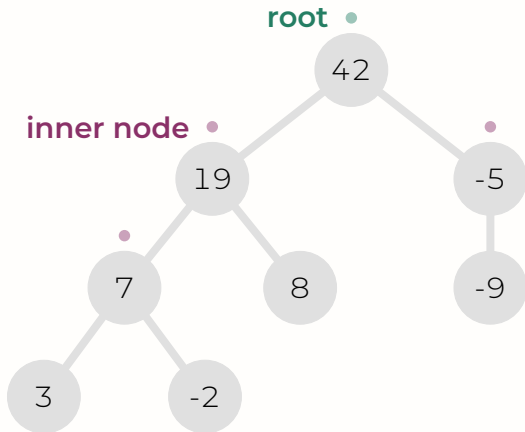
## 4.3

# The Anatomy



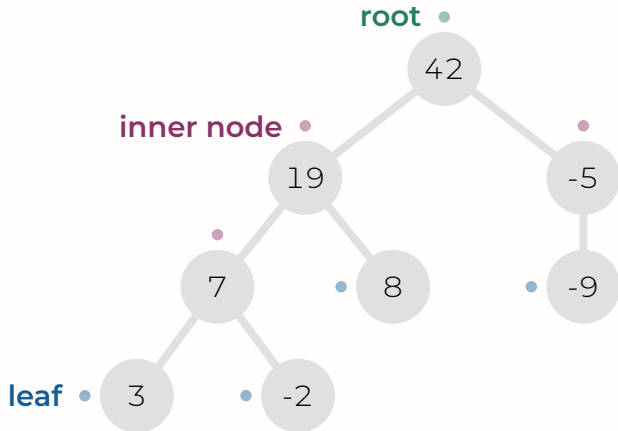
## 4.4

# The Anatomy



## 4.5

# The Anatomy

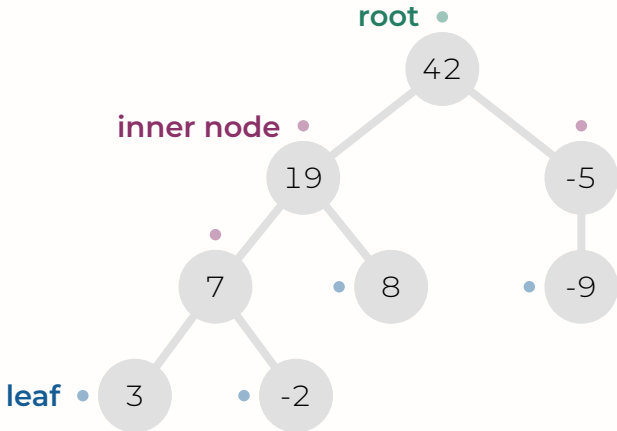




So leaves are nodes without any further child nodes.

## 4.6

# The Anatomy

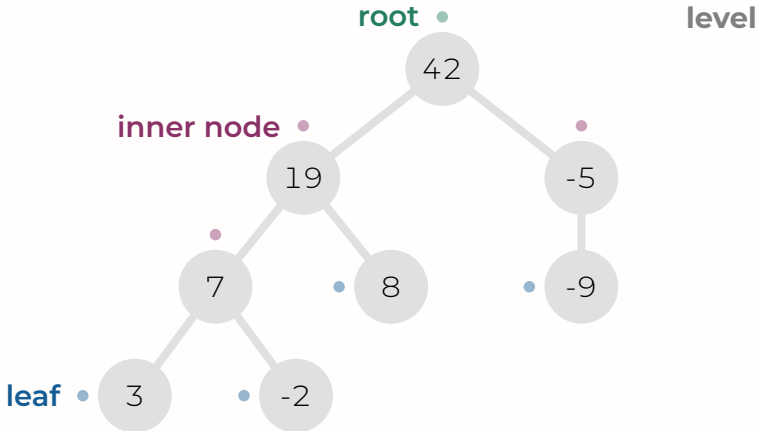




So leaves are nodes without any further child nodes.

## 4.7

# The Anatomy

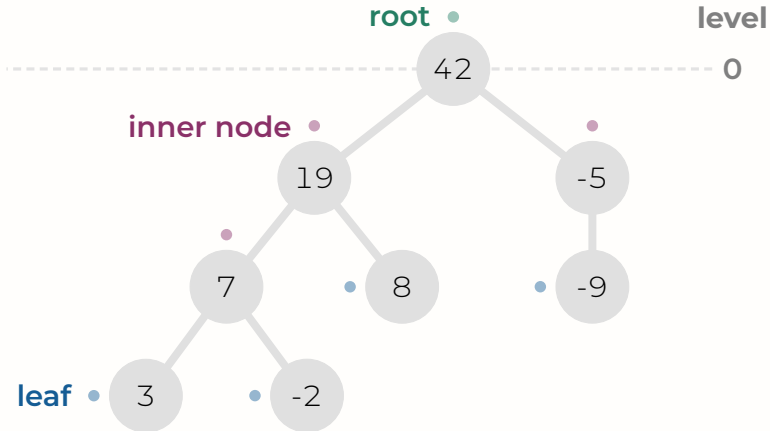




So leaves are nodes without any further child nodes.

## 4.8

# The Anatomy

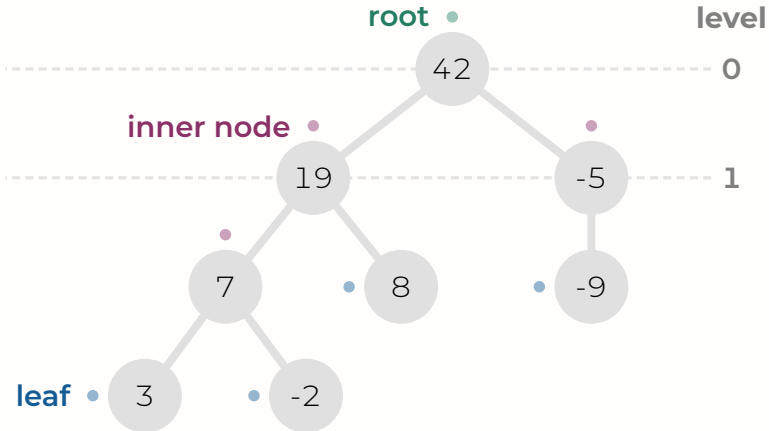




So leaves are nodes without any further child nodes.

## 4.9

# The Anatomy

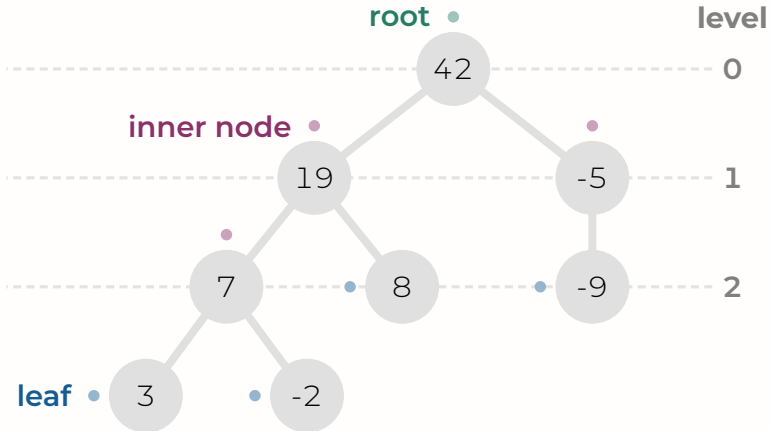




So leaves are nodes without any further child nodes.

## 4.10

# The Anatomy

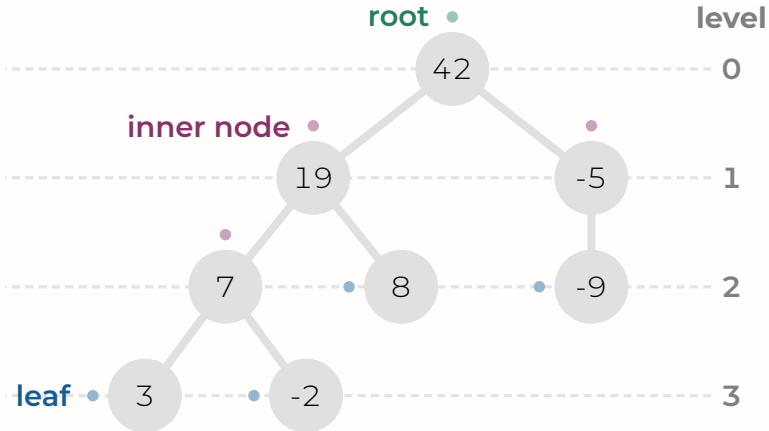




So leaves are nodes without any further child nodes.

## 4.11

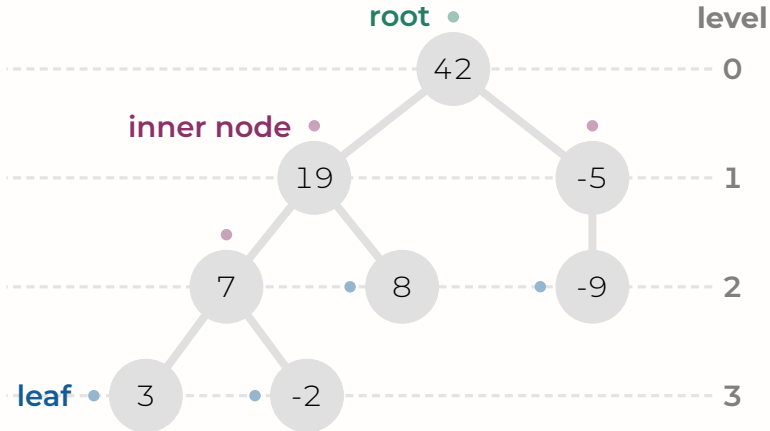
# The Anatomy



## 4.12

## The Anatomy

So leaves are nodes without any further child nodes.



The lines between the nodes are simply called "edges".



## 5.1

# Formalities

[6]: *Episode-Recursion* Sihler, 2021

[2]: *The average height of binary trees and other simple trees*  
Flajolet, 1982



Preliminaries



**Kinds of Heaps**



Operations



Arrays



Wrap-Up



## 5.2

# Formalities

- For us a tree is

[6]: *Episode-Recursion* Sihler, 2021

[2]: *The average height of binary trees and other simple trees*  
Flajolet, 1982



Preliminaries



**Kinds of Heaps**



Operations



Arrays



Wrap-Up



## 5.3

# Formalities

- For us a tree is
  - a graph  $G = (V, E)$  with  $|V|$  nodes and  $|E|$  edges

[6]: *Episode-Recursion* Sihler, 2021

[2]: *The average height of binary trees and other simple trees*  
Flajolet, 1982



Preliminaries



**Kinds of Heaps**



Operations



Arrays



Wrap-Up



## 5.4

# Formalities

- For us a tree is
  - a graph  $G = (V, E)$  with  $|V|$  nodes and  $|E|$  edges
  - $V$  and  $E$  are finite

[6]: *Episode-Recursion* Sihler, 2021

[2]: *The average height of binary trees and other simple trees*  
Flajolet, 1982



Preliminaries



**Kinds of Heaps**



Operations



Arrays



Wrap-Up



## 5.5

# Formalities

- For us a tree is
  - a graph  $G = (V, E)$  with  $|V|$  nodes and  $|E|$  edges
  - $V$  and  $E$  are finite
  - acyclic & connected (“maximally acyclic”)

[6]: *Episode-Recursion* Sihler, 2021

[2]: *The average height of binary trees and other simple trees*  
Flajolet, 1982



Preliminaries



**Kinds of Heaps**



Operations



Arrays



Wrap-Up



## 5.6

# Formalities

- For us a tree is
  - a graph  $G = (V, E)$  with  $|V|$  nodes and  $|E|$  edges
  - $V$  and  $E$  are finite
  - acyclic & connected (“maximally acyclic”)
- Even a binary tree!

[6]: *Episode-Recursion* Sihler, 2021

[2]: *The average height of binary trees and other simple trees*  
Flajolet, 1982



## 5.7

# Formalities

- For us a tree is
  - a graph  $G = (V, E)$  with  $|V|$  nodes and  $|E|$  edges
  - $V$  and  $E$  are finite
  - acyclic & connected (“maximally acyclic”)
- Even a binary tree!
  - Every node has at most two successors

[6]: *Episode-Recursion* Sihler, 2021

[2]: *The average height of binary trees and other simple trees*  
Flajolet, 1982



## 5.8

# Formalities

- For us a tree is
  - a graph  $G = (V, E)$  with  $|V|$  nodes and  $|E|$  edges
  - $V$  and  $E$  are finite
  - acyclic & connected (“maximally acyclic”)
- Even a binary tree!
  - Every node has at most two successors
  - We focus on balanced ones

[6]: *Episode-Recursion* Sihler, 2021

[2]: *The average height of binary trees and other simple trees*  
Flajolet, 1982



## 5.9

# Formalities

A criminally shallow  
look at quite a few cool  
formal foundations.



- For us a tree is
  - a graph  $G = (V, E)$  with  $|V|$  nodes and  $|E|$  edges
  - $V$  and  $E$  are finite
  - acyclic & connected (“maximally acyclic”)
- Even a binary tree!
  - Every node has at most two successors
  - We focus on balanced ones
- Trees can be seen as a recursive structure

[6]: *Episode-Recursion* Sihler, 2021

[2]: *The average height of binary  
trees and other simple trees*  
Flajolet, 1982



## 6.1

# Heapify



Preliminaries



Kinds of Heaps



**Operations**



Arrays

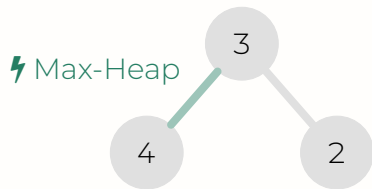


Wrap-Up



## 6.2

# Heapify



## 6.3

# Heapify



## 6.4

# Heapify



- *Up*-heapify: checks leaves  $\rightarrow$  root

## 6.5

# Heapify



- *Up*-heapify: checks leaves  $\rightarrow$  root
- *Down*-heapify: checks root  $\rightarrow$  leaves

## 6.6

# Heapify



Restoring the heap property by swapping!



- *Up-heapify*: checks leaves  $\rightarrow$  root
- *Down-heapify*: checks root  $\rightarrow$  leaves



## 7.1

# Insert/Put



Preliminaries



Kinds of Heaps



**Operations**



Arrays



Wrap-Up



## 7.2

# Insert/ Put

- On the lowest level, as far to the left as possible



## 7.3

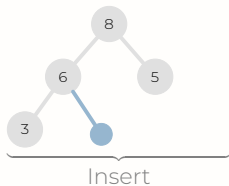
# Insert/ Put

- On the lowest level, as far to the left as possible
- Afterwards: *heapify*

## 7.4

# Insert/Put

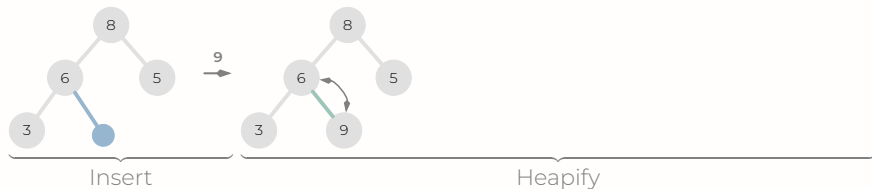
- On the lowest level, as far to the left as possible
- Afterwards: *heapify*



## 7.5

# Insert/Put

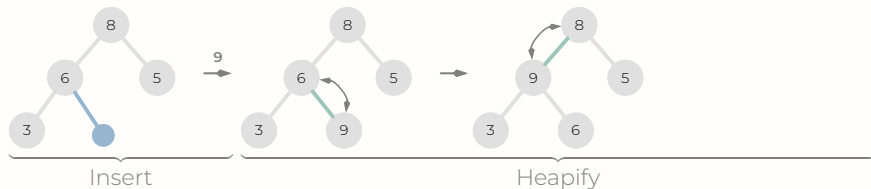
- On the lowest level, as far to the left as possible
- Afterwards: *heapify*



## 7.6

# Insert/Put

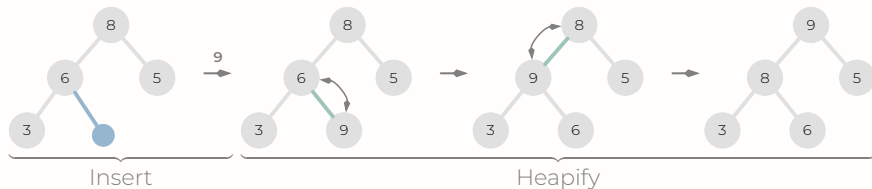
- On the lowest level, as far to the left as possible
- Afterwards: *heapify*



## 7.7

## Insert/Put

- On the lowest level, as far to the left as possible
- Afterwards: *heapify*



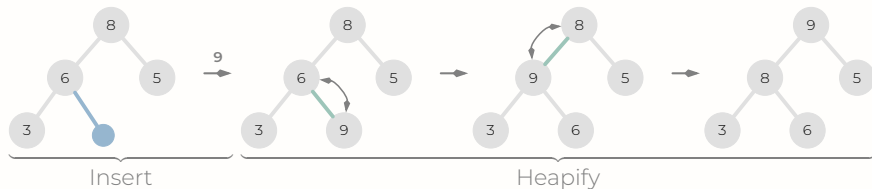
## 7.8

## Insert/Put

If the lowest level is full, we simply create a new one!



- On the lowest level, as far to the left as possible
- Afterwards: *heapify*



## 8.1

# Remove/Get



Preliminaries



Kinds of Heaps



**Operations**



Arrays



Wrap-Up



## 8.2

# Remove/Get

- Replace the topmost element with the “last” one

## 8.3

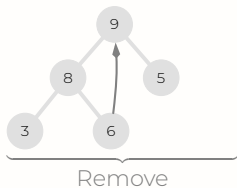
# Remove / Get

- Replace the topmost element with the “last” one
- Afterwards: *heapify*

## 8.4

# Remove/Get

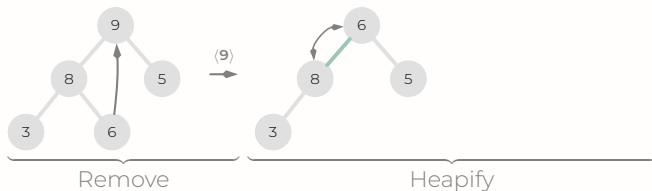
- Replace the topmost element with the “last” one
- Afterwards: *heapify*



## 8.5

## Remove/Get

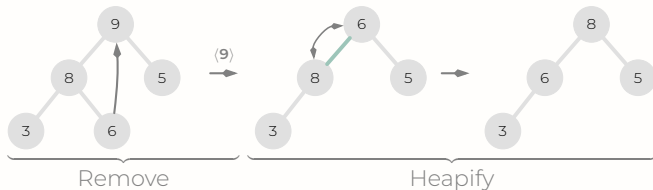
- Replace the topmost element with the “last” one
- Afterwards: *heapify*



## 8.6

# Remove/Get

- Replace the topmost element with the “last” one
- Afterwards: *heapify*



## 9.1

# Further Operations

[3]: *Refined complexity analysis for heap operations* Fredman, 1987

[7]: *Heap: A Data Structure for Efficient Programs* Singh, 2013



Preliminaries



Kinds of Heaps



**Operations**



Arrays



Wrap-Up



## 9.2

# Further Operations

- Find/Extract Min/Max

[3]: *Refined complexity analysis for heap operations* Fredman, 1987

[7]: *Heap: A Data Structure for Efficient Programs* Singh, 2013



Preliminaries



Kinds of Heaps



**Operations**



Arrays



Wrap-Up



## 9.3

# Further Operations

- Find/Extract Min/Max
- Replace (one node with another)

[3]: *Refined complexity analysis for heap operations* Fredman, 1987

[7]: *Heap: A Data Structure for Efficient Programs* Singh, 2013



## 9.4

# Further Operations

- Find/Extract Min/Max
- Replace (one node with another)
- Meld/Join

[3]: *Refined complexity analysis for heap operations* Fredman, 1987

[7]: *Heap: A Data Structure for Efficient Programs* Singh, 2013



## 10.1

# Astonishing Insights



Preliminaries



Kinds of Heaps



Operations



**Arrays**



Wrap-Up



## 10.2

# Astonishing Insights

*Assumptions:*

The heap is completely filled up to the last level.

The last level fills up from the left.

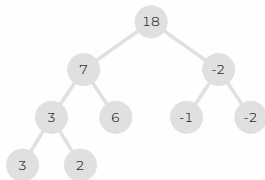


## 10.3

# Astonishing Insights

*Assumptions:*

The heap is completely filled up to the last level.  
The last level fills up from the left.

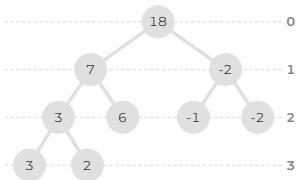


## 10.4

# Astonishing Insights

*Assumptions:*

The heap is completely filled up to the last level.  
The last level fills up from the left.



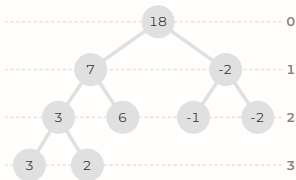
## 10.5

# Astonishing Insights

*Assumptions:*

The heap is completely filled up to the last level.  
The last level fills up from the left.

- Every filled level  $i$  holds  $2^i$  elements



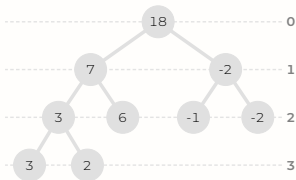
## 10.6

# Astonishing Insights

*Assumptions:*

The heap is completely filled up to the last level.  
The last level fills up from the left.

- Every filled level  $i$  holds  $2^i$  elements
- Numbering by breadth-first traversal allows addressing!



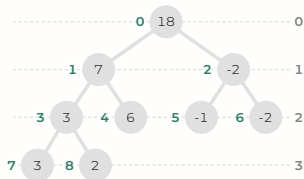
## 10.7

# Astonishing Insights

*Assumptions:*

The heap is completely filled up to the last level.  
The last level fills up from the left.

- Every filled level  $i$  holds  $2^i$  elements
- Numbering by breadth-first traversal allows addressing!



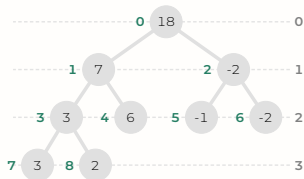
## 10.8

# Astonishing Insights

*Assumptions:*

The heap is completely filled up to the last level.  
The last level fills up from the left.

- Every filled level  $i$  holds  $2^i$  elements
- Numbering by breadth-first traversal allows addressing!
- The left child of  $n$  is  $2 \cdot n + 1$ , the right child  $2 \cdot n + 2$



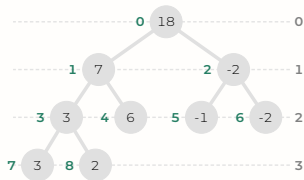
## 10.9

# Astonishing Insights

*Assumptions:*

The heap is completely filled up to the last level.  
The last level fills up from the left.

- Every filled level  $i$  holds  $2^i$  elements
- Numbering by breadth-first traversal allows addressing!
- The left child of  $n$  is  $2 \cdot n + 1$ , the right child  $2 \cdot n + 2$
- The parent of  $n$  is  $\lfloor \frac{n-1}{2} \rfloor$



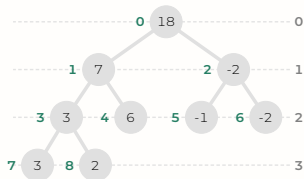
## 10.10

# Astonishing Insights

*Assumptions:*

The heap is completely filled up to the last level.  
The last level fills up from the left.

- Every filled level  $i$  holds  $2^i$  elements
- Numbering by breadth-first traversal allows addressing!
- The left child of  $n$  is  $2 \cdot n + 1$ , the right child  $2 \cdot n + 2$
- The parent of  $n$  is  $\lfloor \frac{n-1}{2} \rfloor$



Hmmmm...



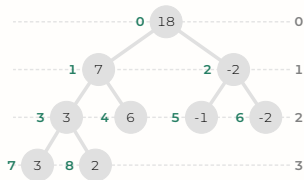
## 10.11

# Astonishing Insights

### *Assumptions:*

The heap is completely filled up to the last level.  
The last level fills up from the left.

- Every filled level  $i$  holds  $2^i$  elements
- Numbering by breadth-first traversal allows addressing!
- The left child of  $n$  is  $2 \cdot n + 1$ , the right child  $2 \cdot n + 2$
- The parent of  $n$  is  $\lfloor \frac{n-1}{2} \rfloor$



Hmmmm... That's just array indexing!



## 11.1

# A Heap as an Array



Preliminaries



Kinds of Heaps



Operations



**Arrays**



Wrap-Up



## 11.2

# A Heap as an Array

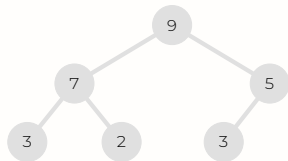
- Heaps usually are “just” arrays



## 11.3

# A Heap as an Array

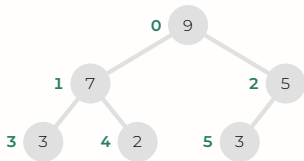
- Heaps usually are “just” arrays



## 11.4

# A Heap as an Array

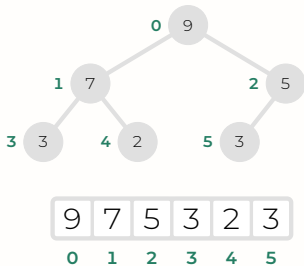
- Heaps usually are “just” arrays



## 11.5

# A Heap as an Array

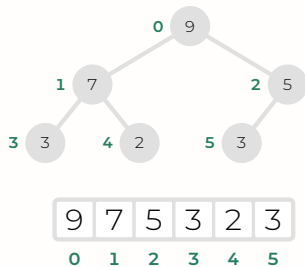
- Heaps usually are “just” arrays



## 11.6

# A Heap as an Array

- Heaps usually are “just” arrays
- Helper routines make working with them pleasant:

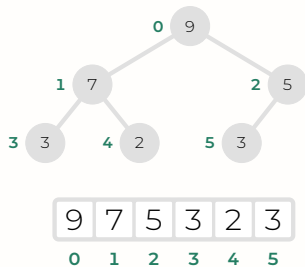


## 11.7

# A Heap as an Array

- Heaps usually are “just” arrays
- Helper routines make working with them pleasant:

```
int left(int n) { return 2 * n + 1; }  
int right(int n) { return 2 * n + 2; }  
int parent(int n) { return (n - 1) / 2; }
```



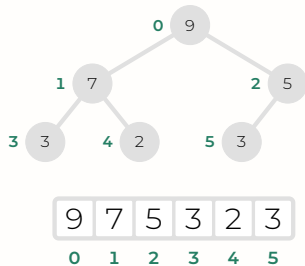
## 11.8

# A Heap as an Array

- Heaps usually are “just” arrays
- Helper routines make working with them pleasant:

```
int left(int n) { return 2 * n + 1; }  
int right(int n) { return 2 * n + 2; }  
int parent(int n) { return (n - 1) / 2; }
```

- Checks certainly do not hurt!



## 11.9

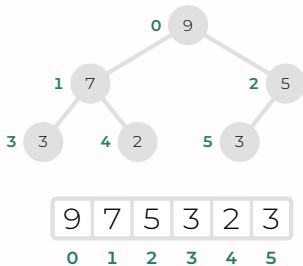
# A Heap as an Array

- Heaps usually are “just” arrays
- Helper routines make working with them pleasant:

```
int left(int n) { return 2 * n + 1; }  
int right(int n) { return 2 * n + 2; }  
int parent(int n) { return (n - 1) / 2; }
```

Tip: with arrays, *swap* is usually extremely helpful as well!

- Checks certainly do not hurt!



## 12.1

# A Descending Example



## 12.2

### A Descending Example

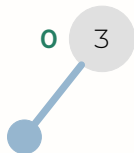
[3, 9, 8, 2, 12]



## 12.3

### A Descending Example

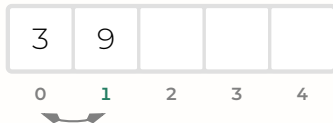
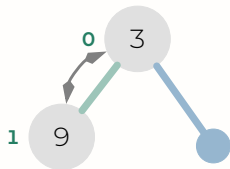
[3, 9, 8, 2, 12]



## 12.4

### A Descending Example

[3, 9, 8, 2, 12]



Preliminaries



Kinds of Heaps



Operations



**Arrays**



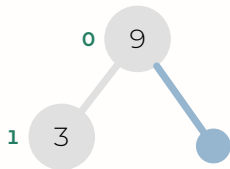
Wrap-Up



## 12.5

### A Descending Example

[3, 9, 8, 2, 12]



Preliminaries



Kinds of Heaps



Operations



**Arrays**



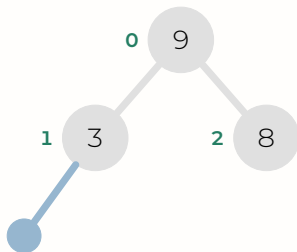
Wrap-Up



## 12.6

### A Descending Example

[3, 9, 8, 2, 12]



Preliminaries



Kinds of Heaps



Operations



**Arrays**



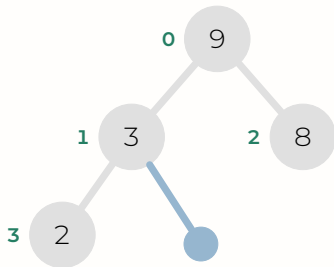
Wrap-Up



## 12.7

### A Descending Example

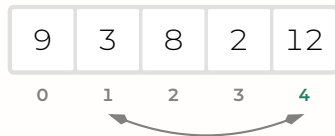
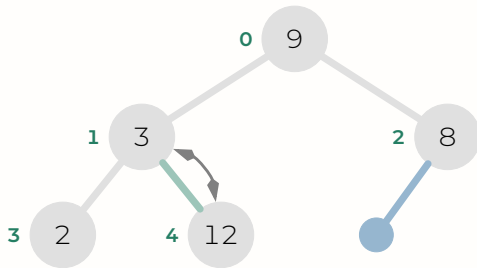
[3, 9, 8, 2, 12]



## 12.8

## A Descending Example

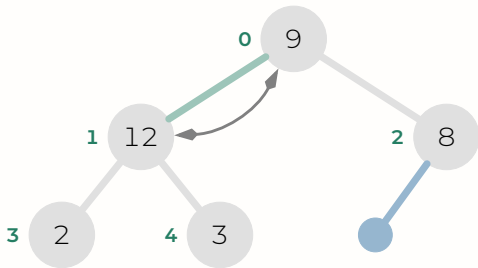
[3, 9, 8, 2, 12]



## 12.9

## A Descending Example

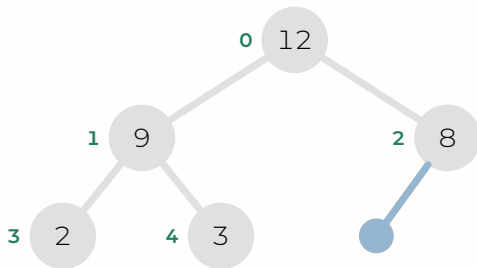
[3, 9, 8, 2, 12]



## 12.10

## A Descending Example

[3, 9, 8, 2, 12]



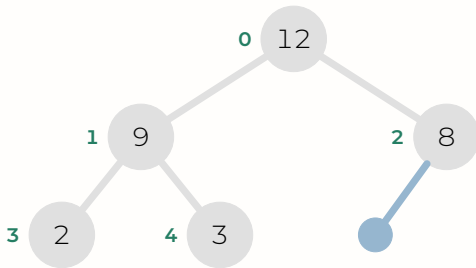
## 12.11

# A Descending Example

If the heap is built with *insert* only, at most the new element violates the heap property. On top of that it stays nicely balanced.



[3, 9, 8, 2, 12]



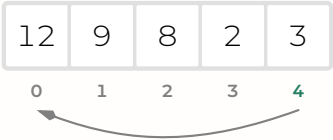
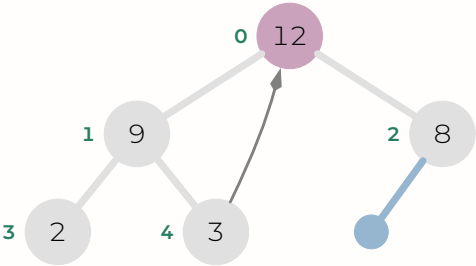


Removal, for a change,  
now with “down-heapify”.

12.12

A Descending Example

[3, 9, 8, 2, 12]



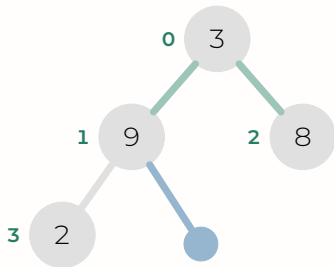
## 12.13

# A Descending Example

Removal, for a change,  
now with “down-heapify”.  
With several conflicts: swap with  
the *more extreme* element.



[3, 9, 8, 2, 12]



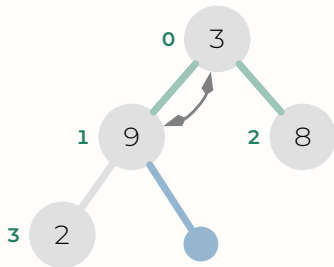
## 12.14

## A Descending Example

Removal, for a change,  
now with “down-heapify”.  
With several conflicts: swap with  
the *more extreme* element.



[3, 9, 8, 2, 12]



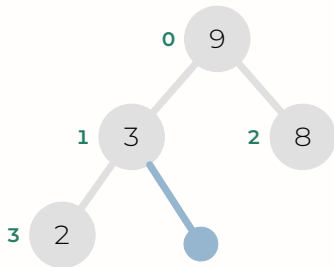
## 12.15

# A Descending Example

Removal, for a change,  
now with “down-heapify”.  
With several conflicts: swap with  
the *more extreme* element.



[3, 9, 8, 2, 12]



## 13.1

## For Practice

[4]: *Binary Search Trees*  
GeeksforGeeks, 2020

[1]: *Exercises for Algorithms and  
Data Structures* Carzaniga, 2016



Preliminaries



Kinds of Heaps



Operations



Arrays



**Wrap-Up**



## 13.2

## For Practice

- Visualize *insert*  $[3, 5, 8, 3, -5, 6]$  (in this order) in a *min*-heap. Best done simultaneously in the tree and in the array.

[4]: *Binary Search Trees*  
GeeksforGeeks, 2020

[1]: *Exercises for Algorithms and  
Data Structures* Carzaniga, 2016



## 13.3

## For Practice

- Visualize *insert*  $[3, 5, 8, 3, -5, 6]$  (in this order) in a *min*-heap. Best done simultaneously in the tree and in the array.
- Now visualize the removal until the heap is empty

[4]: *Binary Search Trees*  
GeeksforGeeks, 2020

[1]: *Exercises for Algorithms and  
Data Structures* Carzaniga, 2016



## 13.4

## For Practice

- Visualize *insert*  $[3, 5, 8, 3, -5, 6]$  (in this order) in a *min*-heap. Best done simultaneously in the tree and in the array.
- Now visualize the removal until the heap is empty
- For these and other data structures, working through them *in practice* helps!

[4]: *Binary Search Trees*  
GeeksforGeeks, 2020

[1]: *Exercises for Algorithms and  
Data Structures* Carzaniga, 2016



- [1] Antonio Carzaniga. “Exercises for Algorithms and Data Structures”. 2016
- [2] Philippe Flajolet and Andrew Odlyzko. “The average height of binary trees and other simple trees”. 1982
- [3] Michael L. Fredman and Thomas H. Spencer. “Refined complexity analysis for heap operations”. 1987
- [4] GeeksforGeeks. *Binary Search Trees*. 2020
- [5] Florian Sihler. *L<sup>A</sup>T<sub>E</sub>X*-Package, *tikzpingus*. 2021
- [6] Florian Sihler. *Episode-Recursion*. 2021
- [7] Sanjay Singh. “Heap : A Data Structure for Efficient Programs”. 2013



**Florian Sihler**

Ulm, September 21, 2026